

Progress Report: Re-Evaluating LLM Loop Invariant Generation & Repair with CeGIS

Kaustubh Srivastava(B23141) Bharat Agrawal(B23341)
Computer Science and Engineering

1 Overview

This report presents a comparative evaluation based on the benchmark study by Akhond et al. [1]. We re-implemented their top-performing invariant synthesis and repair setups using updated model configurations, introduced our new technique, and benchmarked these approaches across both the original dataset and a newly curated dataset.

2 Proposed Technique: CeGIS

- **Failure History:** *In RQ2.2, only the counterexample for most recent failure was provided. So, over multiple turns, the LLM would "forget" earlier failures and oscillate back. So, we pass the entire history of previous failures and attempts in each prompt.*
- **Tabu List:** *We maintain a set of previously failed invariants and if the LLM regenerates something from this set, we make the LLM retry some different invariant(with higher temperature) to prevent falling in an infinite loop.*
- **Syntactic Few Shot Examples:** *As seen from RQ1, providing few shot examples to the LLM helped improve results. So, for the invariant repair problem as well, we provided the LLM with few shot examples based on syntactic similarity.*
- **Anti Overfitting Instructions:** *To prevent the LLM from moving towards reactive point patching and suggesting repairs like $var! = n$ to patch a specific counterexample, we gave it specific instructions to find true general algebraic boundary conditions.*

3 Dataset Curation

We constructed our benchmark set by combining the LoopInvGen dataset [2] (which itself aggregates SyGuS-COMP, Code2Inv [3], ASE FiB [4], SV-COMP [5], and other sources) with the official SyGuS-COMP benchmark archive [6, 7]. After deduplicating problems common to both sources, this yields a total of **1,020** unique problems. Unlike the original LoopInvGen dataset, which consists almost entirely of linear integer arithmetic (LIA) problems, this combined dataset also includes problems from the ALIA (array-based) and NIA (nonlinear arithmetic) theories, making it a more diverse and challenging benchmark for evaluating loop invariant synthesis. We ran our tests on **20** problems from this dataset.

4 Comparative Evaluation

The table below compares the maximum success rates achieved across the original paper baselines [1], our re-implementation on the paper's benchmark, and our evaluation on the newly curated dataset.

Setup	RQ1.1			RQ1.3			RQ1.5		
	$k = 10$	$k = 30$	$k = 50$	$k = 10$	$k = 30$	$k = 50$	$k = 10$	$k = 30$	$k = 50$
Paper Best	27%	41%	49%	61%	72%	76%	57%	73%	78%
Re-Impl.	80.9%	80.9%	80.9%	87.1%	87.1%	87.1%	100%	100%	100%
Curated Dataset (20 Probs)	65%	65%	65%	90%	90%	90%	85%	85%	85%

Table 1: Comparative performance of loop invariant generation capabilities across setups and datasets.

Setup	RQ2.2		CeGIS	
	$Turns = 1$	$Turns = 3$	$Turns = 1$	$Turns = 3$
Paper Best	16%	—	—	—
Re-Impl.	43.3%	71.4%	99.5%	100%
Curated Dataset (20 Probs)	25%	60%	70%	95%

Table 2: Comparative performance of multi-turn loop invariant repair capabilities across setups and datasets.

References

- [1] M. R. Akhond, S. Chakraborty, and G. Uddin, “LLM for loop invariant generation and fixing: How far are we?” *arXiv preprint arXiv:2511.06552*, 2025.
- [2] S. Padhi, R. Sharma, and T. Millstein, “LoopInvGen: A loop invariant generator based on precondition inference,” *arXiv tech report 1707.02029*, 2017.
- [3] X. Si, H. Dai, M. Raghothaman, M. Naik, and L. Song, “Learning loop invariants for program verification,” in *NeurIPS*, 2018.
- [4] S.-W. Lin, J. Sun, H. Xiao, Y. Liu, D. Sanan, and H. Hansen, “FiB: Squeezing loop invariants by interpolation between forward/backward predicate transformers,” in *Proc. ASE*, pp. 793–803, 2017.
- [5] D. Beyer, “Automatic verification of C and Java programs: SV-COMP 2019,” *TACAS Tech. Rep.*, 2019.
- [6] SyGuS-Org, “SyGuS-COMP benchmarks repository,” 2023. [Online]. Available: <https://github.com/SyGuS-Org/benchmarks>
- [7] R. Alur, R. Bodik, G. Juniwal, M. M. Martin, M. Raghothaman, S. A. Seshia, R. Singh, A. Solar-Lezama, E. Torlak, and A. Udupa, “Syntax-guided synthesis,” in *Proc. FMCAD*, IEEE, 2013.